

Especificación de Requisitos de Software (ERS)

Sistema de Mesa de Ayuda (Ticketera)

Proyecto Ticketera Nestjs

22 de septiembre de 2026
Versión 1.0

Documento:	Especificación de Requisitos de Software (ERS)
Norma de referencia:	IEEE 830-1998 (estructura adaptada)
Estado:	Borrador para revisión
Alcance del documento:	Backend completo del sistema (5 microservicios)

Índice

1. Introducción	3
1.1. Propósito	3
1.2. Alcance del documento	3
1.3. Definiciones, acrónimos y abreviaturas	3
1.4. Referencias	3
2. Descripción general	4
2.1. Perspectiva del producto	4
2.2. Funciones del producto	4
2.3. Características de los usuarios	4
2.4. Restricciones generales	4
2.5. Supuestos y dependencias	5
3. Requisitos específicos	5
3.1. Requisitos funcionales	5
3.2. Requisitos no funcionales	7
4. Alcances	9
5. Limitaciones	9

1 Introducción

1.1 Propósito

Este documento especifica los requisitos funcionales y no funcionales del backend del sistema de mesa de ayuda (ticketera) del proyecto **Ticketera Nestjs**. Su objetivo es servir de referencia única y verificable para el equipo de desarrollo, control de calidad, y las revisiones de seguridad, infraestructura y mantenibilidad que ya se han realizado sobre el sistema. Los requisitos aquí descritos reflejan el comportamiento real e implementado del sistema, documentado a partir de la arquitectura, el código y los hallazgos de auditoría registrados en el repositorio.

1.2 Alcance del documento

La especificación cubre exclusivamente el **backend**: los cinco procesos que componen el sistema (**api-gateway**, **bff-web**, **users-service**, **tickets-service**, **notifications-service**) y el servicio de ingesta de correo (**email-ingestion-service**). No especifica el frontend, dado que el proyecto no incluye un cliente propio en el repositorio (solo mockups de referencia pendientes de aprobación para la vista del solicitante).

1.3 Definiciones, acrónimos y abreviaturas

Término	Definición
ERS	Especificación de Requisitos de Software
RF	Requisito Funcional
RNF	Requisito No Funcional
SLA	Service Level Agreement — plazo de atención comprometido para un ticket
BFF	Backend For Frontend — servicio de composición de pantallas
RPC	Remote Procedure Call — invocación entre microservicios vía TCP
JWT	JSON Web Token — mecanismo de autenticación del gateway
ARCO	Derechos de Acceso, Rectificación, Cancelación y Oposición (Ley 21.719, Chile)
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
UoW	Unit of Work — patrón de manejo transaccional
GIN/trgm	Índices de PostgreSQL para búsqueda de texto por similitud
ADMIN, SUPERVISOR, AGENTE, RE-QUESTER	Roles de usuario del sistema, de mayor a menor privilegio

1.4 Referencias

- **CLAUDE.md** — guía de arquitectura y decisiones del repositorio.
- **libs/common/src/contracts/ticket-permissions.ts** — reglas de autorización por rol y estado.
- **docs/proceso/** — planificación, desarrollo, implementación y marcha blanca.
- Ley N.º 21.719 de Protección de Datos Personales (Chile).

- IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*.

2 Descripción general

2.1 Perspectiva del producto

El sistema es un backend de mesa de ayuda construido como un **monorepo NestJS** con cinco procesos independientes que se comunican por transporte TCP de **@nestjs/microservices**, respaldados por una única base de datos PostgreSQL. El **api-gateway** es el único proceso con puerto HTTP expuesto; el resto vive en una red interna sin acceso directo desde fuera.

api-gateway	Borde HTTP: CORS, helmet, rate limit, JWT, Swagger
bff-web	Composición de pantallas para lecturas, sin base de datos propia
users-service	Identidad, roles, derechos ARCO
tickets-service	Dominio de tickets: creación, asignación, SLA, estados
notifications-service	Observador de eventos de dominio, envío de avisos
email-ingestion-service	Conversión de correos entrantes en tickets

2.2 Funciones del producto

En términos generales, el sistema permite: autenticar usuarios; crear tickets manualmente o por correo; clasificar su urgencia automáticamente; calcularles un SLA; asignarlos a un agente según distintas estrategias; registrar comentarios y cambios de estado con una máquina de estados gobernada por rol; notificar a los interesados ante cada cambio; y ejercer los derechos ARCO sobre los datos personales almacenados.

2.3 Características de los usuarios

Rol	Perfil y permisos
REQUESTER	Persona que reporta incidencias. Ve y actúa solo sobre sus propios tickets, con transiciones que dependen del estado de origen.
AGENTE	Resuelve tickets asignados o sin asignar de su especialidad. Comenta y cambia estado dentro de las reglas de autorización.
SUPERVISOR	Visibilidad ampliada: reportes, reasignación, estado de salud del sistema.
ADMIN	Gestión de usuarios (alta, roles, capacidad), configuración y todo lo anterior.

2.4 Restricciones generales

- Toda la interfaz de usuario queda fuera de este backend; no existe cliente propio en el repositorio.
- El sistema debe ejecutarse sobre PostgreSQL 16 con **synchronize** deshabilitado: todo cambio de esquema requiere una migración escrita a mano.
- El transporte entre procesos es TCP interno sin broker de mensajería, por lo que los eventos de integración no son durables.
- El envío saliente real de correos electrónicos no está conectado a un servidor SMTP.

2.5 Supuestos y dependencias

- Se asume una única instancia por servicio en producción (el scheduler de SLA no soporta réplicas sin un lock distribuido adicional).
- Se asume disponibilidad de un buzón IMAP dedicado (`soporte@cmm.uchile.cl`) para la ingesta de correo, con contraseña de aplicación de Google Workspace.
- Se asume que el primer usuario ADMIN se crea por consola antes de exponer el sistema, dado que la API exige un ADMIN autenticado para crear usuarios.

3 Requisitos específicos

3.1 Requisitos funcionales

Los requisitos funcionales se numeran de **RF-01** a **RF-24** y se agrupan por área funcional. Prioridad: **A** (alta / implementada y crítica), **M** (media).

ID	Categoría	Descripción	Prioridad
RF-01	Autenticación	El sistema debe permitir iniciar sesión con correo y contraseña, devolviendo un token de acceso (JWT, 15 min) y un token de refresco (7 días).	A
RF-02	Autenticación	El sistema debe permitir renovar la sesión mediante el token de refresco sin exigir credenciales nuevamente.	A
RF-03	Usuarios	El sistema debe permitir a un ADMIN crear usuarios con rol, habilidades (<i>skills</i>) y capacidad máxima de tickets concurrentes.	A
RF-04	Usuarios	El sistema debe permitir crear el primer usuario ADMIN mediante un comando de consola, sin depender de la API.	A
RF-05	Usuarios	El sistema debe permitir a un ADMIN consultar, actualizar y dar de baja (desactivar) usuarios existentes.	M
RF-06	Tickets	El sistema debe permitir a un usuario autenticado crear un ticket con título, descripción y categoría.	A
RF-07	Tickets	El sistema debe clasificar automáticamente la urgencia del ticket analizando el texto del reporte mediante una heurística de señales de urgencia.	A
RF-08	Tickets	El sistema debe calcular un SLA (plazo de atención) para cada ticket según su prioridad y categoría, aplicando una estrategia de cálculo.	A
RF-09	Tickets	El sistema debe asignar automáticamente el ticket a un agente disponible según una estrategia configurable (por habilidad, por menor carga o round-robin).	A

ID	Categoría	Descripción	Prioridad
RF-10	Tickets	El sistema debe permitir a un SUPERVISOR reasignar manualmente un ticket a otro agente.	M
RF-11	Tickets	La asignación de un ticket al agente que ya lo tiene asignado debe ser una operación idempotente (no generar un nuevo aviso).	A
RF-12	Tickets	El sistema debe permitir cambiar el estado de un ticket únicamente entre las transiciones permitidas por rol y por el estado de origen actual.	A
RF-13	Tickets	El sistema debe permitir agregar comentarios a un ticket, autorizando la acción según el estado real del ticket y el rol del autor.	A
RF-14	Tickets	El sistema debe registrar un historial inmutable de cada cambio de estado del ticket (quién, cuándo, de qué estado a cuál).	A
RF-15	Tickets	El sistema debe permitir buscar y filtrar tickets por estado, vencimiento de SLA, texto libre y rango de fechas de creación.	M
RF-16	Tickets	Los listados de tickets deben soportar paginación y ordenamiento por columnas explícitamente permitidas.	M
RF-17	Notificaciones	El sistema debe notificar al solicitante y al agente asignado (según corresponda) ante cada cambio de estado o comentario nuevo, identificando al interesado real y no a quien ejecutó la acción.	A
RF-18	Notificaciones	El sistema debe degradar el envío de notificaciones a canal in-app cuando el servicio de resolución de destinatarios no esté disponible.	M
RF-19	Ingesta de correo	El sistema debe convertir automáticamente un correo entrante en un ticket nuevo, o en un comentario si el correo es una respuesta a un ticket existente.	A
RF-20	Ingesta de correo	El sistema debe descartar correos que no representen un pedido legítimo de una persona (bucles, remitentes automáticos, dominios no autorizados).	A
RF-21	Ingesta de correo	El sistema debe dar de alta automáticamente, con rol REQUESTER, a un remitente desconocido cuyo dominio esté autorizado.	M
RF-22	Monitoreo	El sistema debe barrer periódicamente los tickets abiertos para detectar incumplimientos de SLA y marcarlos.	A
RF-23	Monitoreo	El sistema debe exponer el estado de los circuitos (dependencias caídas o degradadas) a usuarios con rol SUPERVISOR o ADMIN.	M

ID	Categoría	Descripción	Prioridad
RF-24	Privacidad (ARCO)	El sistema debe permitir a un usuario ejercer los derechos de acceso, rectificación, cancelación, oposición y portabilidad sobre sus datos personales, dejando registro auditable de cada solicitud.	A

3.2 Requisitos no funcionales

Los requisitos no funcionales se numeran de **RNF-01** a **RNF-24**, agrupados por atributo de calidad.

ID	Categoría	Descripción	Prioridad
RNF-01	Seguridad	Toda decisión de autorización que dependa del dato (por ejemplo, si un ticket pertenece al solicitante) debe resolverse en el microservicio dueño del dato, nunca inferirse en el borde.	A
RNF-02	Seguridad	Toda función de autorización debe enumerar explícitamente los casos permitidos y lanzar una excepción en cualquier otro caso; no debe existir una rama final que permita por omisión.	A
RNF-03	Seguridad	Toda comunicación RPC entre microservicios debe ir autenticada con un secreto compartido (RPC_SHARED_SECRET), distinto del secreto de firma de JWT.	A
RNF-04	Seguridad	Los microservicios internos no deben exponer su puerto TCP fuera de la red local del proceso (MICROSERVICE_BIND_HOST).	A
RNF-05	Seguridad	El gateway debe ser el único proceso con acceso a internet y el único que conoce el secreto de firma del JWT; no debe tener acceso directo a la base de datos.	A
RNF-06	Seguridad	Las contraseñas de usuario deben almacenarse con hashing, nunca en texto plano.	A
RNF-07	Seguridad	La documentación interactiva de la API (Swagger) debe poder deshabilitarse por configuración y, en producción, exigir autenticación básica con usuario y contraseña obligatorios.	A
RNF-08	Seguridad	Las búsquedas de texto libre deben tratar el término de búsqueda como parámetro, nunca interpolado en la consulta SQL.	A
RNF-09	Disponibilidad	Toda llamada saliente entre procesos debe protegerse con un circuit breaker que solo cuente fallos de infraestructura contra su umbral, nunca respuestas 4xx legítimas.	A

ID	Categoría	Descripción	Prioridad
RNF-10	Disponibilidad	Un error de autorización o de negocio devuelto por un servicio dependiente no debe transformarse en un error 500 al cruzar otro proceso.	A
RNF-11	Disponibilidad	Los cinco procesos y el servicio de ingesta deben exponer un chequeo de vida (<i>liveness</i>) y uno de disponibilidad (<i>readiness</i>) independientes.	A
RNF-12	Rendimiento	Los tiempos de espera de lock, de sentencia SQL y del circuit breaker deben estar escalonados de menor a mayor, de forma que ninguna capa espere más que la que la contiene.	A
RNF-13	Rendimiento	Ninguna transacción de base de datos debe permanecer abierta mientras se realiza una llamada de red a otro proceso.	A
RNF-14	Rendimiento	Las consultas por atributos de uso frecuente deben poder resolverse usando un índice, verificado con <code>EXPLAIN</code> y <code>enable_seqscan=off</code> .	M
RNF-15	Rendimiento	La resolución de destinatarios de un lote de notificaciones debe realizarse en una única llamada RPC, no una por destinatario.	M
RNF-16	Escalabilidad	El sistema debe documentar explícitamente sus límites de escalado horizontal (resolución fija de <code>host:port</code> en los clientes RPC, límite de <i>throttling</i> por instancia).	M
RNF-17	Mantenibilidad	El dominio de cada microservicio debe organizarse en las capas <code>domain/</code> , <code>application/</code> e <code>infrastructure/</code> , sin que el dominio dependa de <code>@nestjs/microservices</code> ni de HTTP.	A
RNF-18	Mantenibilidad	Las reglas de autorización de tickets deben existir en un único lugar del código, compartido entre el backend y el BFF, para que la interfaz y el caso de uso nunca se desincronicen.	A
RNF-19	Confiabilidad	Debe existir un mecanismo de respaldo de la base de datos verificado y con retención, y un procedimiento de restauración probado que cuente filas por tabla.	A
RNF-20	Confiabilidad	Las migraciones de esquema deben aplicarse mediante un servicio dedicado antes de que arranque cualquier proceso que dependa de la base.	A
RNF-21	Portabilidad	La configuración de cada proceso debe provenir de variables de entorno validadas contra un esquema explícito; una variable no declarada debe impedir el arranque.	A

ID	Categoría	Descripción	Prioridad
RNF-22	Portabilidad	El sistema completo debe poder desplegarse mediante contenedores Docker orquestados por <code>docker compose</code> , con cada servicio recibiendo solo los secretos que efectivamente usa.	A
RNF-23	Usabilidad	Todo error de validación devuelto al cliente debe indicar qué campo específico es inválido y por qué, sin exponer detalles internos del sistema.	M
RNF-24	Cumplimiento normativo	El tratamiento de datos personales debe ajustarse a la Ley 21.719 (Chile): bloqueo temporal de tratamiento, portabilidad y trazabilidad auditable de cada derecho ejercido.	A

4 Alcances

El presente sistema, en su versión especificada por este documento, comprende:

- El backend completo de la mesa de ayuda: los cinco microservicios (`api-gateway`, `bff-web`, `users-service`, `tickets-service`, `notifications-service`) y el servicio de ingesta de correo (`email-ingestion-service`).
- El ciclo de vida completo de un ticket: creación (manual o por correo), clasificación de urgencia, cálculo de SLA, asignación, comentarios, cambios de estado y notificación a los interesados.
- La gestión de usuarios y roles, incluyendo el ejercicio de los derechos ARCO y portabilidad de datos personales según la Ley 21.719.
- Los mecanismos transversales de seguridad, resiliencia (*circuit breaker*), observabilidad y auditoría descritos en los requisitos no funcionales.
- La infraestructura de despliegue mediante contenedores Docker, incluyendo el servicio de migración de base de datos y los scripts de respaldo y restauración.
- Mockups de referencia (no funcionales) para las pantallas de agente/supervisor: login, listado de tickets, detalle de ticket y reportes, en versión web y mobile.

5 Limitaciones

Las siguientes limitaciones son decisiones de diseño ya tomadas y documentadas en el proyecto, no omisiones accidentales. Se listan junto con su costo asumido para que no se re-discutan sin recordar por qué existen.

- **No incluye un frontend propio.** El repositorio no contiene cliente web ni mobile; solo mockups de referencia. La vista del solicitante (rol REQUESTER) está deliberadamente sin diseñar, pendiente de aprobación de los *stakeholders*.
- **Una sola base de datos PostgreSQL** para los cinco procesos, en vez de una por servicio. No es posible escalar o migrar los datos de un servicio de forma independiente del resto.

- **Sin sistema de mensajería intermedio (broker).** Los eventos de integración no son durables: si `notifications-service` está caído en el instante exacto en que ocurre un evento, esa notificación puntual se pierde (el cambio en el ticket queda registrado igual).
- **El envío real de correo saliente no está implementado.** El sistema determina qué correo enviar y a quién, pero el paso final hacia un servidor SMTP queda pendiente de conexión.
- **Sin revocación inmediata de sesión.** El token de acceso es la fuente de verdad durante sus 15 minutos de vida; desactivar un usuario no invalida una sesión ya emitida hasta que expira.
- **El *scheduler* de SLA asume una única instancia por proceso.** Con réplicas activas del mismo servicio, dos procesos barrerían los tickets en paralelo sin coordinación adicional.
- **El secreto RPC es único y estático,** no por par de servicios: autentica al llamador pero no lo identifica individualmente. Un servicio comprometido puede invocar a cualquier otro del grupo interno.
- **No hay *WAL archiving*.** El punto de recuperación (RPO) efectivo es el intervalo entre respaldos manuales, no minutos.
- **Los respaldos se almacenan en el mismo disco que la base de datos,** sin copia remota automatizada.
- **El escalado horizontal de los servicios internos no reparte carga.** El cliente RPC de cada proceso resuelve `host:port` una única vez y mantiene un socket persistente; agregar réplicas sin un balanceador TCP delante no distribuye tráfico.
- **El límite de *rate limiting* es por instancia,** no compartido entre réplicas del gateway.
- **La suite de pruebas end-to-end (`test:e2e`) está declarada pero no implementada;** la cobertura actual es de pruebas unitarias y de integración parcial.
- **Alcance de la auditoría de privacidad (ARCO):** un hallazgo de auditoría pendiente documenta que el bloqueo temporal de privacidad aún no es consultado por el resolver de asignación ni por el procesamiento de notificaciones — el campo se persiste correctamente pero todavía no detiene el tratamiento real de datos en esos dos puntos.